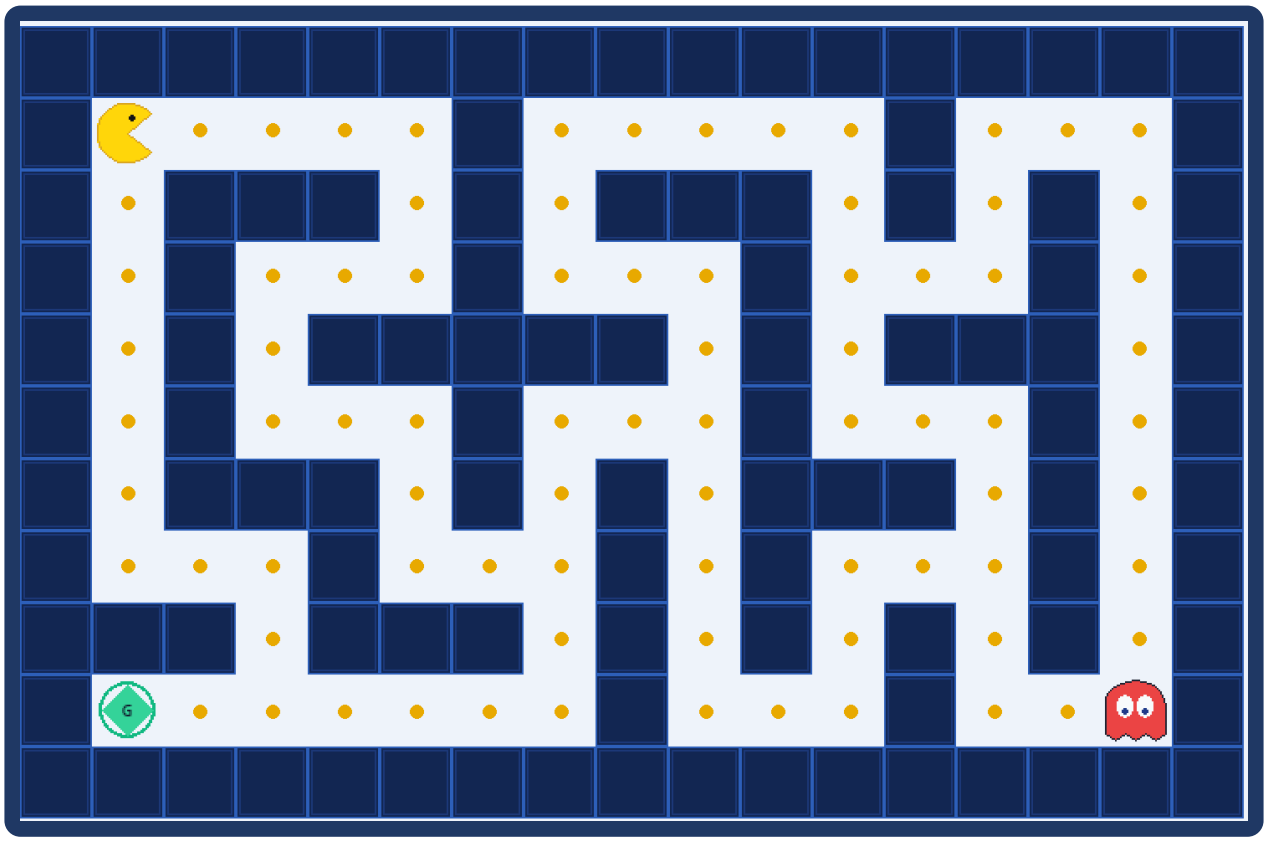
ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ (UET)



REPORT

PACMAN GAME

NHÓM 07

Hà Nội, 2026

MỤC LỤC

1.Phát biểu bài toán..... 2

2. Biểu diễn trạng thái và không gian tìm kiếm.....2

2.1. Trạng thái (lớp GameState).....2

2.2. Bài toán một mục tiêu (PMSearchProblem).....3

2.3. Bài toán nhiều mục tiêu (multi_goal_search.py).....3

2.4. Bài toán đối kháng.....3

3. Kiến trúc hệ thống và thiết kế giao diện.....4

3.1. Kiến trúc phân tầng.....4

3.2. Luồng dữ liệu..... 4

3.3. Giao diện dòng lệnh (main.py).....5

3.4. Giao diện đồ họa Pygame (ui/).....5

4. Thiết kế các thuật toán..... 6

5. Thiết kế và lập luận về heuristic.....7

5.1. Heuristic cho bài toán một mục tiêu: khoảng cách Manhattan.....7

5.2. Heuristic cho bài toán nhiều mục tiêu.....7

5.3. Phương pháp so sánh..... 7

6. Thiết kế Minimax, Alpha–Beta và evaluation function.....8

6.1. Minimax (minimax.py).....8

6.2. Alpha–Beta (alpha_beta.py).....8

6.3. Bài toán đối kháng và khoảng cách mê cung (adv_problem.py).....8

6.4. Hàm đánh giá evaluate(state, depth)..... 9

7. Thiết lập thực nghiệm.....10

7.1. Bản đồ..... 10

7.2. Chỉ số đo.....10

7.3. Cách thực hiện..... 10

8. Kết quả và phân tích.....12

8.1. So sánh thuật toán tìm đường.....12

8.2. Đánh giá heuristic..... 15

8.3. Đánh giá tìm kiếm đối kháng.....16

9. Phần phát triển của nhóm.....17

10. Hạn chế.....18

11. Kết luận.....18

12. Phân công công việc và khai báo sử dụng AI.....19

BÁO CÁO PHÂN CÔNG CÔNG VIỆC VÀ ĐÓNG GÓP CỦA CÁC THÀNH VIÊN.....19

1.Phát biểu bài toán

Nhóm xây dựng một hệ thống Pacman đơn giản bằng Python. Pacman di chuyển trên bản đồ dạng lưới có tường, thức ăn, điểm đích (Goal) và Ghost. Hệ thống giải quyết hai lớp bài toán:

- Lập kế hoạch đường đi: Pacman tìm đường tới một mục tiêu, tới một trong nhiều Goal, hoặc thu thập toàn bộ thức ăn.
- Ra quyết định đối kháng: Pacman chọn hành động khi Ghost cũng di chuyển để săn đuổi, với Minimax và Alpha–Beta Pruning.

Môi trường đáp ứng các yêu cầu tối thiểu của đề bài:

- Bản đồ lưới đọc từ file văn bản, kích thước thay đổi được; có thể nạp nhiều bản đồ khác nhau mà không sửa mã nguồn.
- Ký hiệu bản đồ: # tường, . thức ăn, P Pacman, G đích, @ Ghost, khoảng trắng là ô đi được, ~ là ô nước cost 2, S là ô bùn cost 3 .
- Bốn hành động di chuyển: UP, DOWN, LEFT, RIGHT; có kiểm tra hành động hợp lệ (không đâm tường, không ra ngoài biên).
- Trạng thái kết thúc: PLAYING, WIN, LOSE trong chế độ đối kháng.
- Giao diện đồ họa Pygame có chọn thuật toán, chọn bản đồ, bắt đầu, tạm dừng, chạy từng bước, khởi động lại, kèm bảng chỉ số và nhật ký.

Chương trình hỗ trợ 7 thuật toán: DFS, BFS, UCS, Greedy Best-First, A*, Minimax và Alpha–Beta.

2. Biểu diễn trạng thái và không gian tìm kiếm

2.1. Trạng thái (lớp GameState)

GameState là một ảnh chụp hoàn chỉnh của trò chơi, gồm:

Thuộc tính	Ý nghĩa
pacman_position	Tọa độ Pacman (row, col)
ghost_positions	Danh sách tọa độ các Ghost
food	Tập hợp (set) tọa độ thức ăn còn lại
score	Điểm số hiện tại
game_map	Ma trận bản đồ tĩnh
status	PLAYING, WIN hoặc LOSE

Các thao tác chính:

- Hành động hợp lệ: check_valid_position(position) trả về False nếu ra ngoài biên hoặc ô là #, độ phức tạp O(1).
- Hàm chuyển trạng thái: move(action) tính vị trí mới; nếu không hợp lệ trả về None, ngược lại sao chép tập food, nếu ô mới có thức ăn thì xóa khỏi tập và cộng 10 điểm, rồi trả về một GameState mới.

- Tính bất biến: move không sửa đối tượng cũ, nên các nhánh tìm kiếm không làm hỏng dữ liệu của nhau.
- Băm được: __eq__ so sánh vị trí Pacman, vị trí Ghost, food và score; __hash__ chuyển ghost_positions thành tuple và food thành frozenset. Nhờ đó trạng thái đưa được vào set để kiểm tra đã thăm trong O(1).

2.2. Bài toán một mục tiêu (PMSearchProblem)

- Trạng thái ban đầu: Pacman ở vị trí xuất phát spawn.
- Hành động / hàm chuyển: expand_pacman_moves(state) thử 4 hướng, với mỗi hướng hợp lệ trả về bộ ba (next_state, action, step_cost).
- Chi phí bước:
  - Ô trống mặc định bằng 1
  - Có thể thay bằng hàm chi phí tùy biến cost_fn trong map chi phí di chuyển không đồng nhất: ~/S là 2/3 .
- Điều kiện đích: Pacman đứng tại ô target.
- Chi phí đường đi: measure_path_cost(actions) cộng chi phí thực tế của chuỗi hành động.
- Heuristic: heuristic(state) dùng heuristic_fn nếu được truyền vào, mặc định là khoảng cách Manhattan.

2.3. Bài toán nhiều mục tiêu (multi_goal_search.py)

- Trạng thái ban đầu: GameState(spawn, [], set(food), 0, game_map).
- Biểu diễn mục tiêu đã thu thập: chính là tập food còn lại trong GameState; ăn một viên thì viên đó bị loại khỏi tập.
- Điều kiện hoàn thành (reached_target):
  - Trả về False nếu food còn phần tử
  - Nếu đã hết thức ăn mà bài toán có Goal cuối thì Pacman phải đứng tại Goal
  - Nếu không có Goal cuối thì hoàn thành.
- Kích thước không gian:
  - Gồm vị trí Pacman và tập con thức ăn còn lại, cỡ N × 2^K với N là số ô đi được và K là số thức ăn.
  - Khi K lớn (ví dụ K > 12), số trạng thái vượt 10^9, vì vậy nhóm bổ sung cách phân rã ở mục 9.

2.4. Bài toán đối kháng

- Tác tử:
  - Tác tử 0 là Pacman (nút MAX)
  - Các tác tử còn lại là Ghost (nút MIN).

- Chuyển trạng thái trong mô phỏng: step(pacman_action, target) thực hiện theo thứ tự: Pacman đi; nếu đâm vào Ghost thì LOSE; nếu tới đích thì WIN; sau đó từng Ghost lần lượt đi; sau mỗi lần Ghost đi, nếu Ghost bắt được Pacman thì LOSE.
- Utility: được cho bởi hàm đánh giá (mục 6); thắng và thua có giá trị ±100000 điều chỉnh theo độ sâu.

3. Kiến trúc hệ thống và thiết kế giao diện.

3.1. Kiến trúc phân tầng

Tầng	Thư mục / file	Trách nhiệm
Khởi chạy	main.py, test_part.py, requirements.txt	Điều phối CLI/UI, kiểm thử nhanh, khai báo thư viện
Tiện ích bản đồ	src/utils/	Thẩm định, đọc file, bóc tách tọa độ
Mô hình thế cờ	src/pacman/game_state.py	GameState bất biến, băm được
Tác tử Ghost	src/ghost/ghost.py	Hành vi săn mồi của Ghost
Tìm kiếm cổ điển	src/search/	DFS, BFS, UCS, Greedy, A*, nhiều mục tiêu
Đối kháng	src/logic_pacman_vs_ghost/	Bài toán đối kháng, Minimax, Alpha–Beta
Giao diện	ui/	Giao diện Pygame
Kiểm thử	tests/	30 test tự động
Dữ liệu	maps/	Bộ bản đồ thực nghiệm

Thư viện cần cài (requirements.txt): pygame >= 2.6.0 cho đồ họa 2D và vòng lặp thời gian thực.

3.2. Luồng dữ liệu

1. load_map(file_path) đọc file bằng UTF-8, bỏ ký tự xuống dòng kiểu Windows/Linux, cắt các dòng trống cuối file, rồi gọi check_map. Nếu file không tồn tại hoặc bản đồ không hợp lệ, hàm ghi cảnh báo và trả về None thay vì làm chương trình dừng đột ngột.
2. check_map kiểm tra: bản đồ không rỗng; các dòng cùng độ dài (ma trận chữ nhật); chỉ chứa ký tự trong #.PG@~S ; có đúng 1 P; có ít nhất 1 mục tiêu (G hoặc .).
3. get_map_info(game_map, return_foods=False) quét bản đồ và trả về vị trí Pacman, danh sách Goal, danh sách Ghost (và danh sách thức ăn nếu return_foods=True). Nếu bản đồ chỉ có . mà không có G và return_foods=False, hàm coi danh sách thức ăn là danh sách Goal để tương thích ngược.
4. Dữ liệu được đưa vào bài toán tìm kiếm (PMSearchProblem, bài toán nhiều mục tiêu hoặc AdvProblem), chạy bởi thuật toán tương ứng. Kết quả được hiển thị ở CLI hoặc UI.

CLI và UI dùng chung các lớp thuật toán trong src/, nên số liệu hiển thị là kết quả chạy thật của thuật toán, không phải dữ liệu tạo riêng cho phần trình diễn.

3.3. Giao diện dòng lệnh (main.py)

- python main.py chạy menu tương tác:
  - Chọn một trong 8 bản đồ từ maps/adversarial_small.txt đến maps/bad_map.txt hoặc nhập đường dẫn
  - Chọn thuật toán (1: DFS, 2: BFS, 3: UCS, 4: Greedy, 5: A*, 6: Minimax, 7: Alpha–Beta).
- Nhóm thuật toán cổ điển (1–5) phân nhánh giữa Single-Goal, Multi-Goal và Multi-Food.
- Nhóm đối kháng (6–7) cho nhập độ sâu và chọn chế độ 1 (tính nước đi đầu tiên) hoặc chế độ 2 (mô phỏng cả trận bằng vòng lặp state.step()).
- in_ban_do(...) in mê cung ra terminal bằng ký tự ASCII: P Pacman, @ Ghost, G đích, . thức ăn, # tường.
- python main.py --ui chuyển sang giao diện đồ họa.

3.4. Giao diện đồ họa Pygame (ui/)

Cửa sổ có độ phân giải 1260 × 780, khóa 60 FPS (clock.tick(60)), gồm 4 khu vực: khung mê cung, bảng điều khiển (Controls), bảng thống kê (Metrics, 8 thông số thời gian thực) và bảng nhật ký (Log).

File	Vai trò
theme.py	Bảng màu và định dạng; Pacman màu vàng, 6 màu Ghost, không Ghost nào màu vàng
components.py	Widget: Button, ButtonGroup (chọn thuật toán, chế độ), NumberStepper (Depth từ 1 đến 6), MapSelector (lật trang file map), ScrollableLogBox (nhật ký có con lăn chuột và thanh trượt)
renderer.py	Tính kích thước ô linh hoạt (16–48 px) và căn giữa mê cung; vẽ Pacman, Ghost bằng vector; vẽ vệt đường đi
controller.py	Điều phối trạng thái và chạy thuật toán ở luồng nền
main.py	Vòng lặp Pygame chính, xử lý sự kiện chuột và phím

Minh họa đường đi: draw_path_trail() vẽ vệt mờ các ô đã đi cùng đường neon có mũi tên chỉ hướng; controller ghi lại mảng path_trail.

Đa luồng: Thuật toán chạy trong threading.Thread(target=self._run_classic_search, daemon=True) để giao diện không bị treo. Ba sự kiện đồng bộ điều khiển luồng nền:

- _stop_event: kích hoạt khi Reset hoặc đổi bản đồ, luồng nền thoát an toàn.
- _pause_event: khi Pause thì xóa cờ, luồng nền chờ.
- _step_trigger: khi bấm STEP, luồng chạy đúng một bước rồi dừng.

Controller chuyển trạng thái READY khi nạp bản đồ hợp lệ, ERROR khi gặp bản đồ lỗi (không crash), WIN khi tìm thấy đích.

4. Thiết kế các thuật toán.

Tất cả thuật toán tìm kiếm cổ điển dùng PMSearchProblem và trả về danh sách hành động, hoặc None khi không có lời giải. Số trạng thái mở rộng được ghi vào expanded_count.

Thuật toán	Cấu trúc dữ liệu	Tối ưu	Độ phức tạp	Ghi chú thiết kế
DFS	Stack (list, pop() kiểu LIFO), phần tử (state, actions)	Không	Thời gian O(b^m), bộ nhớ O(b×m) (b là hệ số nhánh, m là độ sâu tối đa)	Kiểm tra if state in visited: continue trước khi mở rộng
BFS	deque, popleft() O(1)	Có (số bước)	Thời gian: O(b^d), Bộ nhớ: O(b^d). (b là hệ số nhánh, d là độ sâu của đích có ít bước nhất)	Đánh dấu visited sớm, ngay khi đưa vào hàng đợi, tránh trùng lặp trong hàng đợi
UCS	heapq, phần tử (new_cost, counter, next_state, new_actions)	Có (chi phí)	Thời gian: O(b^(1 + [C*/ε])), Bộ nhớ: O(b^(1 + [C*/ε])) . (với C* là chi phí tối ưu, ϵ là chi phí bước đi nhỏ nhất; khi chi phí đồng nhất sẽ tương đương O(b^d),với d là độ sâu của đích có ít bước nhất)	counter tự tăng để phá hòa, tránh so sánh hai đối tượng GameState gây TypeError
Greedy	heapq theo h(n)	Không	Thời gian: O(b^m), Bộ nhớ: O(b^m). (Trường hợp xấu nhất tương đương DFS; phụ thuộc lớn vào chất lượng hàm heuristic h(n))	Ít node nhưng đường đi có thể khúc khuỷu
A*	heapq theo f(n) = g(n) + h(n)	Có (khi heuristic admissible và consistent)	Thời gian: O(b^(1 + [C*/ε])). Bộ nhớ: O(b^(1 + [C*/ε])) (trường hợp xấu nhất khi h=0; thực tế giảm sâu còn khoảng O(b*^d) tùy thuộc vào độ chính xác của h, với d là độ sâu của nghiệm tối ưu và b* là hệ số nhánh hiệu dụng)	new_g = g + move_cost, new_h = heuristic(next_state), new_f = new_g + new_h

Chu trình, ngõ cụt, vô nghiệm:

- Mọi thuật toán dùng tập visited nên không lặp vô hạn trên bản đồ có chu trình; ngõ cụt chỉ làm hàng đợi/ngăn xếp cạn nhánh.
- Khi bản đồ không có đường tới đích (ví dụ no_solution_map.txt), các thuật toán trả về None.

Nhiều mục tiêu:

- Bài toán gom thức ăn dùng cùng các thuật toán trên không gian trạng thái ở mục 2.3.
- Với bản đồ nhiều thức ăn, main.py còn có chế độ Multi-Food phân rã (mô tả ở mục 9).

Tác tử Ghost (ghost.py, choose_action) gồm ba tầng bảo hiểm:

1. BFS né Ghost đồng đội: coi vị trí các Ghost khác là vật cản, nên khi một Ghost chặn hành lang thì Ghost kia tìm đường vòng, tạo thế gọng kìm.

2. BFS nới lỏng: nếu tầng 1 không tìm được đường, bỏ tập cấm nhưng bước đầu tiên vẫn ưu tiên không đè lên Ghost khác.
3. Greedy Manhattan: nếu cả hai tầng BFS đều không tới được Pacman (ví dụ Pacman bị tường bao kín), chọn nước đi giảm khoảng cách Manhattan nhiều nhất; Ghost không bao giờ đứng yên (trừ khi không có hành động hợp lệ hoặc đã trùng vị trí Pacman).

5. Thiết kế và lập luận về heuristic.

5.1. Heuristic cho bài toán một mục tiêu: khoảng cách Manhattan

h(n) = |row − target_row| + |col − target_col|, cài đặt trong PMSearchProblem.heuristic. Lập luận với chi phí bước mặc định bằng 1 (lưới 4 hướng):

- Không âm: là tổng hai giá trị tuyệt đối nên h(n) ≥ 0.
- Giá trị tại đích: khi Pacman đứng tại đích thì hai hiệu bằng 0, nên h = 0.
- Admissible: mỗi bước di chuyển đổi đúng một tọa độ đi 1 đơn vị, nên cần ít nhất |dr| + |dc| bước; tường chỉ có thể làm đường thực tế dài hơn. Do đó h(n) ≤ h*(n) (chi phí thực tế tối thiểu).
- Consistent: với mỗi bước n → n' chi phí c, khoảng cách Manhattan thay đổi tối đa 1, tức h(n) ≤ 1 + h(n') ≤ c + h(n') khi c ≥ 1.

Lưu ý: lập luận admissible và consistent chỉ giữ khi mọi chi phí bước ≥ 1. Nếu dùng cost_fn có chi phí nhỏ hơn 1 thì Manhattan không còn bảo đảm các tính chất này.

5.2. Heuristic cho bài toán nhiều mục tiêu

h(n) = max trên các thức ăn còn lại của khoảng cách Manhattan từ Pacman tới thức ăn đó (max_dist_to_food).

- Không âm: là giá trị lớn nhất của các số không âm.
- Admissible: để ăn hết thức ăn, Pacman bắt buộc phải tới được viên xa nhất, nên cần ít nhất từng đó bước.
- Consistent (lập luận của nhóm): sau một bước, với mỗi viên còn lại f', khoảng cách Manhattan từ vị trí cũ không quá 1 + khoảng cách từ vị trí mới, nên giá trị max từ vị trí cũ không vượt quá 1 + h(n'). Nếu bước đó ăn viên e đang đứng ngay cạnh thì d(n, e) = 1 ≤ 1 + h(n'). Do vậy h(n) ≤ c + h(n').
- Giá trị tại đích: Khi hết thức ăn và không có Goal cuối, reached_target trả về True, nên h = 0. Khi có Goal cuối, trạng thái đích là hết thức ăn và p = G. Lúc đó reached_target cũng trả về True nên h = 0.

5.3. Phương pháp so sánh

Theo yêu cầu đề bài, nhóm so sánh A* với heuristic do nhóm đề xuất và A* với heuristic bằng 0:

+ Khi h = 0, A* suy biến thành UCS: test test_a_star_zero_heuristic_equivalence_to_ucs kiểm chứng số node mở rộng và chi phí giống hệt UCS.

+ Phần phân tích khi nào heuristic có lợi nằm ở mục 8.2.

6. Thiết kế Minimax, Alpha–Beta và evaluation function.

6.1. Minimax (minimax.py)

Hàm _minimax_value(state, depth, agent):

- Dừng khi is_terminal(state) hoặc depth <= 0, trả về evaluate(state, depth).
- Nút MAX (agent 0, Pacman): lấy giá trị lớn nhất trên các trạng thái kế tiếp.
- Nút MIN (Ghost): lấy giá trị nhỏ nhất.
- Chuyển lượt bằng get_next_agent; depth chỉ giảm 1 khi lượt quay lại Pacman, nghĩa là một tầng độ sâu là một vòng chơi đầy đủ (Pacman và tất cả Ghost).
- Tại gốc cây có phạt lặp vị trí: trừ count * 150 nếu vị trí kế tiếp nằm trong recent_positions.

6.2. Alpha–Beta (alpha_beta.py)

Cùng cấu trúc với Minimax, thêm hai biên alpha, beta:

- Alpha: Giá trị tốt nhất mà tác tử MAX được đảm bảo đạt được tính tới thời điểm hiện tại (cận dưới).
- Beta: Giá trị tốt nhất mà tác tử MIN được đảm bảo đạt được tính tới thời điểm hiện tại (cận trên).
- Nút MAX: cập nhật best và alpha; nếu beta <= alpha thì cắt (Beta cutoff).
- Nút MIN: cập nhật best và beta; nếu beta <= alpha thì cắt (Alpha cutoff).
- Số node có thể giảm từ O(b^d) xuống xấp xỉ O(b^(d/2)) trong trường hợp thứ tự nước đi tốt nhất.

Vì cắt tỉa chỉ bỏ các nhánh không thể ảnh hưởng đến giá trị gốc, Alpha–Beta cho cùng hành động và cùng giá trị với Minimax khi cùng trạng thái, độ sâu và hàm đánh giá; nhóm kiểm chứng bằng test ở mục 8.3.

6.3. Bài toán đối kháng và khoảng cách mê cung (adv_problem.py)

AdvProblem cung cấp is_terminal, get_successors_for_agent, get_next_agent và evaluate. Hàm get_maze_distance(pos1, pos2) dùng BFS tính trước khoảng cách thực giữa các ô và lưu vào cache tĩnh _MAZE_DIST_CACHE (khóa theo bản đồ), nên mỗi lần tra cứu trong hàm đánh giá là O(1).

6.4. Hàm đánh giá evaluate(state, depth)

Tiêu chí	Điều kiện	Điểm
Bị bắt	check_pacman_caught()	−100000 − depth × 1000
Thắng	Pacman tại đích	+100000 + depth × 1000
Cơ sở	Khoảng cách mê cung tới đích	−dist_goal × 40
Ghost kề sát, không lối thoát	min_ghost_dist <=1 và không có ô kề nào (không tường, không Ghost) gần đích hơn vị trí hiện tại	−50000
Ghost kề sát, còn lối thoát	min_ghost_dist <=1 và có ít nhất một ô kề (không tường, không Ghost) gần đích hơn vị trí hiện tại	−500
Nguy hiểm cao	min_ghost_dist = 2	- 15000
Đang bị tiếp cận	min_ghost_dist = 3	−4000
An toàn	min_ghost_dist ≥ 4	-1000
Ghost đuổi sau	ghost_to_goal > dist_goal và chasing_dist ≥ 2	+ min(chasing_dist, 6) × 10

Ghi chú:

- Các dòng phạt/thưởng liên quan đến Ghost chỉ áp dụng khi bản đồ có ít nhất một Ghost, và chúng cộng dồn với điểm cơ sở.
- min_ghost_dist là khoảng cách mê cung (BFS) tới Ghost gần nhất. blocking_dist và chasing_dist là khoảng cách tới Ghost gần nhất trong từng nhóm tương ứng.
- Khi không có Ghost nào thuộc nhóm “đuổi sau”, chasing_dist = + vô cùng và code vẫn cộng +60. Đây là hành vi thực tế của code hiện tại.

Lập luận thiết kế:

- Dùng khoảng cách mê cung thực tế (BFS) thay vì đường chim bay để không bị đánh lừa bởi tường.
- Phạt phi tuyến theo khoảng cách tới Ghost gần nhất: càng gần càng bị phạt rất nặng, đẩy Pacman ưu tiên né Ghost trước khi tiến tới đích.
- Thưởng/phạt kết thúc có thành phần depth × 1000. Vì depth truyền vào là độ sâu còn lại, trạng thái thắng sớm được thưởng nhiều hơn và trạng thái thua sớm bị phạt nặng hơn.
- Phạt vị trí lặp tại gốc (150 điểm mỗi lần) giảm hiện tượng Pacman đi qua đi lại (áp dụng riêng tại nút gốc trong Minimax/Alpha-Beta, không nằm trong evaluate)

7. Thiết lập thực nghiệm.

7.1. Bản đồ

File	Kích thước	Đặc điểm / mục đích
single_small_few_solvable.txt	9×15	Map nhỏ không gian thoáng, 1 Goal; làm cơ sở đối sánh tính tối ưu và số node mở rộng của BFS, DFS, UCS và A*.
multi_goal_test.txt	9×15	Bản đồ có 4 vị trí Goal (G) ở các góc khác nhau; kiểm thử bài toán gom hết mục tiêu (Multi-Goal Search).
multi_large_many_solvable.txt	31x41	Bản đồ lớn nhiều vật cản gồm 5 điểm thức ăn (.) và 1 đích thoát hiểm (G); kiểm tra giới hạn chịu tải của A* đa mục tiêu.
adversarial_small.txt	11x21	Bản đồ có hành lang hẹp dài, 1 Ghost bám sát sau lưng, 2 Food và 1 Goal; kiểm tra khả năng né tránh của Minimax và Alpha-Beta.
large_map.txt	22x19	Bản đồ lớn với buồng giam 3 Ghost ở trung tâm; đo mức bùng nổ node và hiệu quả cắt tỉa của Alpha-Beta (giảm 82% trạng thái) so với Minimax.
weighted_swamp_shortcut_dilemma.txt	6x25	Map có đường thẳng ngắn (22 bước) nhưng xuyên qua bãi nước (~) (cost: 2) và đầm lầy (S) (cost: 3); kiểm tra UCS/A* tự động chọn đường vòng trên khô (26 bước) để tối ưu tổng chi phí so với BFS.
weighted_multi_food_wetlands.txt	13x25	Map kết hợp 8 viên thức ăn (.), 1 đích (G) cùng các vùng nước (~) (cost: 2) và đầm lầy (S) (cost: 3); kiểm tra Multi-Goal UCS/A* tối ưu hóa lộ trình gom hết mục tiêu trên địa hình chi phí không đồng nhất.
dead_end_maze.txt	11x17	Bản đồ có nhiều ngõ cụt sâu và vòng lặp kèm 1 Ghost; kiểm tra hàm đánh giá (evaluation function) tránh bị dồn vào đường cụt.
no_solution_map.txt	9x9	Bản đồ nhỏ có đường tới Goal bị tường bịt kín hoàn toàn; kiểm tra thuật toán dừng đúng và trả về None an toàn.
bad_map.txt	4x7	Dòng không đều và có ký tự lạ; kiểm tra bộ thẩm định bản đồ

Bộ bản đồ bao phủ: nhỏ và lớn,ít và nhiều vật cản, ngõ cụt, có và không có lời giải, một mục tiêu và nhiều mục tiêu, di chuyển đồng nhất và di chuyển không đồng nhất.

7.2. Chỉ số đo

- Tìm đường: có tìm thấy lời giải hay không, số bước, tổng chi phí, số trạng thái mở rộng (expanded_count), thời gian chạy.
- Đối kháng: tác tử, độ sâu, hành động được chọn, giá trị đánh giá, số trạng thái đã xét, thời gian ra quyết định, kết quả trận.

7.3. Cách thực hiện

- Chạy bằng python main.py (terminal) hoặc python main.py --ui (giao diện); test_part.py là script kiểm thử nhanh gọi DFS, BFS, UCS, A* trên maps/multi_large_many_solvable.txt.
- Benchmark :chạy python benchmark.py từ thư mục gốc của dự án. Script tự chạy 4 thực nghiệm, in kết quả ra màn hình và lưu vào thư mục results/ (tự tạo nếu chưa có).

  - So sánh thuật toán tìm kiếm cổ điển:
    - Chạy DFS, BFS, UCS, Greedy, A* trên 6 bản đồ gồm 4 bản đồ đơn mục tiêu (single_small_few_solvable, dead_end_maze, large_map, no_solution_map) và 2 bản đồ đa mục tiêu (multi_goal_test, multi_large_many_solvable).
    - Mỗi dòng kết quả gồm: có tìm được lời giải hay không, số bước, tổng chi phí, số nút mở rộng và thời gian (ms). Bản đồ vô nghiệm kiểm tra việc thuật toán dừng đúng và báo không có đường đi.
  - Chi phí đồng nhất và không đồng nhất:
    - Chạy cả 5 thuật toán trên weighted_swamp_shortcut_dilemma (đơn mục tiêu) và weighted_multi_food_wetlands (đa mục tiêu), mỗi bản đồ ở hai chế độ chi phí.
    - Chế độ đồng nhất tính mọi ô bằng 1.
    - Chế độ không đồng nhất tính ô ~ bằng 2, ô S bằng 3 và các ô còn lại bằng 1.
    - Thực nghiệm này cho thấy BFS tối ưu theo số bước nhưng không tối ưu theo chi phí khi địa hình có trọng số, còn UCS và A* vẫn tối ưu.
  - Đánh giá heuristic:
    - So sánh số nút mở rộng của UCS, A*(h = 0) và A* với heuristic đề xuất (mục 5) trên 7 trường hợp, gồm 2 bản đồ có trọng số.
    - Cột Reduction (%) là mức giảm số nút của A*(h) so với UCS, tính bằng (1 − nút A*(h) / nút UCS) × 100.
  - Đối kháng Minimax và Alpha–Beta:
    - Chạy trên adversarial_small (1 ma), dead_end_maze (1 ma) và large_map (3 ma) với độ sâu giới hạn 1, 2, 3.
    - Với mỗi cặp bản đồ và độ sâu, script kiểm tra hai thuật toán cho cùng hành động và cùng giá trị (MATCH hoặc MISMATCH).
    - Script cũng báo số nút, thời gian, tỉ lệ cắt tỉa và hệ số tăng tốc của Alpha–Beta so với Minimax.
- Bộ kiểm thử tự động gồm 30 test ở tests/:
  - test_map_loader.py: nạp bản đồ hợp lệ, bản đồ lỗi, file không tồn tại, trích tọa độ, bản đồ lớn, bản đồ vô nghiệm.
  - test_search_classic.py: 5 thuật toán, đa mục tiêu, thức ăn kèm Goal cuối, bản đồ vô nghiệm, tương đương A* h=0 và UCS.
  - test_adversarial.py: tương đương Minimax/Alpha–Beta, hiệu quả cắt tỉa, mô phỏng trận, né va chạm.
  - test_ui.py: quét bản đồ, controller, quy tắc màu, vệt đường đi.

8. Kết quả và phân tích.

8.1. So sánh thuật toán tìm đường

Các kết quả đã được kiểm thử tự động xác nhận:

- Trên adversarial_small.txt, BFS, UCS và A* đều trả về đường đi tối ưu 16 bước; DFS và Greedy tìm được đường hợp lệ về đích.
- Trên bản đồ không có lối thoát, cả 5 thuật toán trả về None, không lặp vô hạn.
- BFS giải được bài toán nhiều Goal trên multi_goal_test.txt; Pacman cũng giải được trường hợp ăn sạch thức ăn rồi về Goal cuối.

Bảng 1. Chi phí bước đồng nhất (cost = 1)

Bản đồ	Thuật toán	Tìm thấy	Số bước	Tổng chi phí	Node mở rộng	Thời gian (ms)
single_small_few_solvable	DFS	Có	50	50	51	0,35
	BFS	Có	18	18	79	0,86
	UCS	Có	18	18	79	0,74
	Greedy	Có	18	18	19	0,12
	A*	Có	18	18	79	0,53
dead_end_maze	DFS	Có	24	24	25	0,12
	BFS	Có	12	12	25	0,12
	UCS	Có	12	12	25	0,12
	Greedy	Có	12	12	13	0,06
	A*	Có	12	12	15	0,23
large_map	DFS	Có	85	85	180	1,62
	BFS	Có	23	23	175	1,51
	UCS	Có	23	23	175	1,48
	Greedy	Có	23	23	26	0,27
	A*	Có	23	23	65	0,41
no_solution_map	DFS	Không	N/A	N/A	6	0,06
	BFS	Không	N/A	N/A	6	0,04
	UCS	Không	N/A	N/A	6	0,18
	Greedy	Không	N/A	N/A	6	0,06
	A*	Không	N/A	N/A	6	0,05
multi_goal_test	DFS	Có	65	65	71	0,39
	BFS	Có	27	27	256	1,26
	UCS	Có	27	27	256	2,55
	Greedy	Có	27	27	113	0,86
	A*	Có	27	27	110	0,89
multi_large_many_solvable	DFS	Có	584	584	1027	7,68
	BFS	Có	188	188	19229	135,42
	UCS	Có	188	188	19229	144,03
	Greedy	Có	248	248	9186	92,37

Bản đồ	Thuật toán	Tìm thấy	Số bước	Tổng chi phí	Node mở rộng	Thời gian (ms)
	A*	Có	188	188	14896	149,25

Bảng 2. Chi phí đồng nhất (mọi ô = 1) và không đồng nhất (~ = 2, S = 3)

Bản đồ	Thuật toán	Số bước	Tổng chi phí	Node mở rộng	Thời gian (ms)
swamp_shortcut (đồng nhất)	DFS	22	22	23	0,12
	BFS	22	22	52	0,24
	UCS	22	22	52	0,25
	Greedy	22	22	23	0,55
	A*	22	22	23	0,15
swamp_shortcut (không đồng nhất)	DFS	22	54	23	0,10
	BFS	22	54	52	0,26
	UCS	26	26	50	0,63
	Greedy	22	54	23	0,17
	A*	26	26	37	0,31
multi_wetlands (đồng nhất)	DFS	224	224	297	1,80
	BFS	76	76	17169	105,96
	UCS	76	76	17169	169,82
	Greedy	154	154	17448	201,51
	A*	76	76	4645	56,80
multi_wetlands (không đồng nhất)	DFS	224	263	297	1,90
	BFS	76	85	17169	108,02
	UCS	78	82	16971	166,65
	Greedy	154	164	17448	207,13
	A*	78	82	5039	82,06

Phân tích xu hướng (đã đối chiếu với thiết kế):

- Tính tối ưu:
  - BFS, UCS, A* luôn cho chi phí tối ưu trên các bản đồ chi phí 1 (18, 12, 23, 27, 188).
  - DFS cho đường hợp lệ nhưng dài hơn 2 đến 3,7 lần (ví dụ 85 so với 23 trên large_map, 584 so với 188 trên multi_large_many_solvable).
- Bùng nổ node:
  - BFS và UCS không có thông tin dẫn hướng nên mở rộng nhiều node nhất: 19 229 node (khoảng 135 đến 144 ms) trên multi_large_many_solvable, trong khi DFS chỉ cần 1 027 node (7,68 ms).

  - Trên large_map, BFS/UCS mở rộng 175 node, A* chỉ 65 node. DFS tốn ít bộ nhớ nhưng không phải lúc nào cũng ít node nhất: trên large_map DFS mở rộng 180 node, nhiều hơn BFS (175).
- Greedy:
  - Thường mở rộng rất ít node (19, 13, 26 node trên ba bản đồ đầu) và tối ưu trên các bản đồ đơn giản.
  - Tuy nhiên nó không bảo đảm tối ưu: 248 bước so với 188 (+31,9%) trên multi_large_many_solvable, và 154 so với 76 (gấp đôi) trên multi_wetlands.
  - Trên multi_wetlands, Greedy còn mở rộng 17 448 node, nhiều hơn cả BFS (17 169) và chậm nhất (201,51 ms ở chế độ đồng nhất), nên nhận định “Greedy luôn ít node” chỉ đúng với bản đồ đơn giản.
- Chi phí đồng nhất bằng 1:
  - BFS và UCS cho cùng số bước, cùng chi phí và cùng số node trên mọi bản đồ (Bảng 1 và Bảng 2: 52 node trên swamp_shortcut, 17 169 node trên multi_wetlands), tức UCS suy biến thành BFS.
  - A* vẫn tối ưu (22 và 76) nhưng mở rộng ít node hơn hẳn: 23 so với 52 (giảm 55,8%) và 4 645 so với 17 169 (giảm 72,9%).
- Chi phí không đồng nhất:
  - Đường ít bước nhất không còn là đường rẻ nhất.
  - Trên swamp_shortcut, DFS, BFS và Greedy đi thẳng qua dải ô lầy (22 bước, cost 54), còn UCS và A* đi vòng 26 bước nhưng chỉ tốn cost 26, rẻ hơn khoảng 2,1 lần.
  - Trên multi_wetlands, UCS và A* đi nhiều hơn 2 bước (78 so với 76) nhưng cost 82, thấp hơn BFS (85), Greedy (164) và DFS (263).
  - Chỉ UCS và A* bảo đảm tối ưu theo chi phí; BFS chỉ tối ưu theo số bước.
- Ảnh hưởng của chi phí lên từng thuật toán:
  - DFS, BFS và Greedy không dùng chi phí khi tìm nên đi đúng cùng một đường ở hai chế độ (cùng số bước, cùng số node), chỉ có chi phí báo cáo tăng.
  - UCS và A* đổi đường đi nên số node thay đổi:
    - A* tăng node khi có chi phí không đồng nhất (23 lên 37, 4 645 lên 5 039) vì Manhattan chỉ tính 1 mỗi bước, ước lượng thấp hơn chi phí thật nên heuristic lỏng hơn.
    - Mức giảm node của A* so với UCS thu hẹp từ 55,8% xuống 26,0% trên swamp_shortcut, và từ 72,9% xuống 70,3% trên multi_wetlands.

8.2. Đánh giá heuristic

Bảng 3. UCS so với A(h=0) so với A(h)**

Bản đồ	UCS (node)	A* h=0 (node)	A* h (node)	Giảm node so với UCS	Thời gian UCS / A* h (ms)
single_small_few_solvable	79	79	79	0,0%	0,60 / 0,78
dead_end_maze	25	25	15	40,0%	0,24 / 0,12
large_map	175	175	65	62,9%	1,10 / 0,52
swamp_shortcut	50	50	37	26,0%	0,50 / 0,32
multi_goal_test	256	256	110	57,0%	2,38 / 0,99
multi_large_many_solvable	19229	19229	14896	22,5%	183,94 / 160,68
multi_wetlands	16971	16971	5039	70,3%	232,12 / 94,80

Phân tích xu hướng:

- A* với h = 0 so với UCS:
  - Số node bằng nhau trên cả 7 bản đồ, tức A* suy biến đúng thành UCS.
  - Chênh lệch thời gian giữa hai cột (ví dụ 232,12 so với 304,32 ms trên multi_wetlands) chỉ là nhiễu đo, vì số node giống hệt nhau.
- A* với Manhattan:
  - Vẫn tối ưu (đã lập luận admissible và consistent ở mục 5)
  - Chi phí bằng UCS ở mọi bản đồ, và giảm 22,5% đến 70,3% số node trên 6 trong 7 bản đồ.
- Khi nào heuristic có lợi:
  - Rõ nhất khi không gian rộng hoặc chi phí không đồng nhất:
    - multi_wetlands giảm 70,3% (232,12 xuống 94,80 ms)
    - large_map giảm 62,9%
    - multi_goal_test giảm 57,0%.
- Khi nào lợi ích không rõ:
  - Ở bản đồ nhỏ single_small_few_solvable, A*(h) mở rộng đúng 79 node như UCS (giảm 0%) và còn chậm hơn (0,78 so với 0,60 ms) do chi phí tính heuristic.
  - Giả thuyết “Manhattan bỏ qua tường nên khó cắt node trong mê cung ngõ cụt” chưa được số liệu ủng hộ: dead_end_maze vẫn giảm 40% (25 xuống 15 node). Cần nói thận trọng rằng ở bản đồ nhỏ này mức giảm tuyệt đối chỉ 10 node.
  - Khi chi phí không đồng nhất, Manhattan ước lượng thấp hơn chi phí thật nên lợi ích thu hẹp (mức giảm trên swamp_shortcut từ 55,8% xuống 26,0%, xem 8.1).
- Nhiều mục tiêu:
  - Mức giảm thấp nhất xuất hiện ở multi_large_many_solvable (22,5%, tức 19 229 xuống 14 896 node), phù hợp với nhận định heuristic max-Manhattan khá lỏng khi thức ăn rải rác.

  - Tuy vậy multi_goal_test vẫn giảm 57,0%, nên mức lỏng còn phụ thuộc vào cách bố trí thức ăn trên từng bản đồ.

8.3. Đánh giá tìm kiếm đối kháng

Các kết quả đã được kiểm thử tự động xác nhận:

- Tại độ sâu 1, 2 và 3, Minimax và Alpha–Beta trả về cùng hành động và cùng giá trị đánh giá; tương đương này vẫn đúng khi bật phạt vị trí lặp (recent_positions).
- Trên bản đồ lớn hơn, Alpha–Beta mở rộng ít node hơn hẳn Minimax.
- Trong mô phỏng trận thật trên bản đồ nhỏ, Pacman dùng Alpha–Beta chiến thắng; Pacman không tự đâm vào Ghost nếu có lối thoát khác.

Bảng 4. Số liệu đo (Minimax so với Alpha–Beta)

Bản đồ	Độ sâu	Tác tử	Hành động	Số trạng thái xét	Thời gian (ms)	Giảm node	Kết quả trận
adversarial_small (1 Ghost)	1	Minimax	UP	12	0,80		—
	1	Alpha–Beta	UP	11	0,10	8,3%	—
	2	Minimax	UP	78	1,46		—
	2	Alpha–Beta	UP	55	0,30	29,5%	—
	3	Minimax	UP	557	3,41		—
	3	Alpha–Beta	UP	250	1,32	55,1%	—
dead_end_maze (1 Ghost)	1	Minimax	DOWN	6	0,35		—
	1	Alpha–Beta	DOWN	5	0,05	16,7%	—
	2	Minimax	DOWN	30	0,51		—
	2	Alpha–Beta	DOWN	21	0,07	30,0%	—
	3	Minimax	DOWN	126	0,95		—
	3	Alpha–Beta	DOWN	65	0,40	48,4%	—
large_map (3 Ghosts)	1	Minimax	DOWN	24	1,26		—
	1	Alpha–Beta	DOWN	16	0,07	33,3%	—
	2	Minimax	DOWN	384	3,24		—
	2	Alpha–Beta	DOWN	152	0,57	60,4%	—
	3	Minimax	DOWN	6684	35,76		—
	3	Alpha–Beta	DOWN	1200	6,98	82,0%	—

Phân tích:

- Tăng theo độ sâu:
  - Số node Minimax tăng nhanh theo độ sâu (large_map: 24, 384, 6 684, hệ số khoảng ×16 đến ×17 mỗi mức), nên Alpha–Beta càng sâu càng tiết kiệm.
  - Mức giảm tăng dần ở cả ba bản đồ: 8,3%, 29,5%, 55,1% (adversarial_small); 16,7%, 30,0%, 48,4% (dead_end_maze); 33,3%, 60,4%, 82,0% (large_map).
- Ảnh hưởng của hệ số nhánh:

  - Bản đồ có 3 Ghost (hệ số nhánh lớn nhất) được cắt tỉa nhiều nhất

=> Ở độ sâu 3, thời gian giảm từ 35,76 xuống 6,98 ms (khoảng 5,1 lần).

- So với giới hạn lý thuyết O(b^(d/2)):
  - Nếu Minimax tăng ×17 mỗi mức thì lý tưởng Alpha–Beta chỉ tăng khoảng √17 ≈ ×4,1.
  - Thực tế đo trên large_map là ×9,5 (16 sang 152) rồi ×7,9 (152 sang 1 200), tức chưa đạt giới hạn lý thuyết.
  - Tương tự adversarial_small: Minimax ×6,5 và ×7,1, Alpha–Beta ×5,0 và ×4,5 (lý tưởng khoảng ×2,6).
  - Nguyên nhân hợp lý là thứ tự duyệt nước đi chưa tối ưu, vì giới hạn lý thuyết chỉ đạt khi nước tốt nhất luôn được duyệt trước.
- Trường hợp khác dự đoán:
  - Ở độ sâu 1, Alpha–Beta chỉ bớt 1 đến 8 node (mức giảm nhỏ về tuyệt đối).
  - Thời gian đo ở độ sâu thấp không phản ánh đúng mức cắt tỉa: ví dụ dead_end_maze độ sâu 2 chỉ giảm 30% node nhưng thời gian giảm từ 0,51 xuống 0,07 ms, nhiều khả năng do nhiễu đo và khởi động lần chạy đầu của Minimax.

9. Phần phát triển của nhóm.

Các mở rộng có mã nguồn trong dự án:

- Evaluation function cải tiến (mục 6.4):
  - Hàm đa tiêu chí với phạt phi tuyến theo khoảng cách Ghost, phạt ngõ cụt, thưởng/phạt kết thúc phụ thuộc độ sâu, phạt lặp vị trí tại gốc.
  - Được kiểm tra bởi test_pacman_avoids_immediate_ghost_collision, test_pacman_wins_on_small_map_simulation và bản đồ dead_end_maze.txt.
- Tối ưu thời gian: cache khoảng cách mê cung BFS _MAZE_DIST_CACHE giúp tra cứu khoảng cách trong O(1) khi hàm đánh giá được gọi rất nhiều lần.
- Phân rã bài toán nhiều thức ăn (plan_multi_food_path):
  - Thay vì giải đồng thời không gian N × 2^K, hàm lặp tham lam: chọn thức ăn gần nhất theo Manhattan, dùng thuật toán đã chọn (DFS, BFS, UCS, Greedy hoặc A*) tìm đường tới đó, nối các chặng hành động, rồi lặp lại.
  - Độ phức tạp giảm từ cấp số nhân xuống khoảng O(K × N); thời gian thực thi khoảng 0.05 giây. Đổi lại, đường đi tổng thể không bảo đảm tối ưu.
- Ghost nhiều tầng bảo hiểm (mục 4):
  - BFS né Ghost đồng đội
  - BFS nới lỏng
  - Fallback Manhattan, giúp nhiều Ghost phối hợp truy đuổi.

- Trực quan hóa quá trình di chuyển: vệt đường đi neon, chế độ STEP, nhật ký và 8 chỉ số thời gian thực.
- Bộ thẩm định bản đồ (check_map) và bản đồ lỗi bad_map.txt bảo đảm nạp bản đồ an toàn.

10. Hạn chế.

- DFS và Greedy không bảo đảm đường đi tối ưu.
- Chế độ Multi-Food phân rã là tham lam theo khoảng cách Manhattan nên không tối ưu toàn cục; nếu một chặng không có đường đi thì vòng lặp dừng (break) và chỉ trả về phần kế hoạch đã có.
- Không gian trạng thái của bài toán nhiều mục tiêu nguyên bản tăng theo N × 2^K, nên chỉ phù hợp với số thức ăn nhỏ.
- Heuristic max-Manhattan cho nhiều mục tiêu khá lỏng và Manhattan bỏ qua tường.
- Hàm đánh giá của Minimax và Alpha–Beta dùng các hệ số cố định do nhóm đặt tay (20, 2, 150, −50000…), chưa được tinh chỉnh tự động hay đánh giá đầy đủ trên mọi bản đồ.
- Hành vi Ghost trong mô phỏng là một chính sách truy đuổi theo BFS đã định sẵn (3 tầng), chưa có Ghost ngẫu nhiên hay nhiều kiểu hành vi khác nhau.
- Giao diện hiển thị đường đi và chỉ số, nhưng chưa có trực quan hóa riêng cho quá trình mở rộng trạng thái.

11. Kết luận

Dự án Pacman AI Search (Nhóm 07) đã hoàn thành các yêu cầu của học phần:

- 7 thuật toán cốt lõi:
  - DFS
  - BFS
  - UCS
  - Greedy
  - A^*
  - Minimax,
  - AlphaBeta Pruning
- Giải quyết bài toán:
  - Đơn/đa mục tiêu
  - Chi phí di chuyển đồng nhất/không đồng nhất,
  - Đối kháng
- Hệ thống benchmark kiểm thử tự động
- Pass 30 Unit Tests bao phủ từ tiền xử lý, thuật toán, đối kháng đến giao diện người dùng.

- Giao diện terminal và UI trực quan có thể tương tác

12. Phân công công việc và khai báo sử dụng AI.

BÁO CÁO PHÂN CÔNG CÔNG VIỆC VÀ ĐÓNG GÓP CỦA CÁC THÀNH VIÊN

Dự án 1 – Pacman: Tìm kiếm và trò chơi đối kháng

Môn học: Biểu diễn tri thức và tìm kiếm nâng cao (UET.AI2014_5)

Đơn vị: Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội (UET-VNU)

Nhóm thực hiện: Nhóm 07

1. Danh sách thành viên

STT	Họ và tên	Mã sinh viên	Vai trò
1	Bùi Chí Thành	25023046	Thành viên
2	Khúc Hữu Trung	25023062	Thành viên
3	Vũ Thành Thanh Tùng	25023040	Thành viên
4	Lê Anh Vũ	25023077	Trưởng nhóm

2. Phân công công việc cá nhân

2.1. Bùi Chí Thành

- Xây dựng cấu trúc ban đầu của dự án theo chuẩn hướng đối tượng (OOP).
- Viết lớp quản lý trạng thái trò chơi trong `game_state.py` và tổ chức luồng chạy chính trong `main.py`.
- Bổ sung chi phí di chuyển cho các loại địa hình: Nước (~) có chi phí 2 và Đầm lầy (`S`) có chi phí 3.
- Tổng hợp nội dung và viết file `CONTRIBUTIONS.md`.

Trạng thái: Hoàn thành.

2.2. Khúc Hữu Trung

- Xây dựng lớp mô hình hóa bài toán đối kháng trong `adv_problem.py`, bao gồm trạng thái trò chơi và cơ chế luân phiên lượt đi giữa Pacman và Ghost.
- Cài đặt hai thuật toán Minimax và Alpha-Beta Pruning.
- Tổng hợp nội dung và viết báo cáo đồ án trong file `report.pdf`.

Trạng thái: Hoàn thành.

2.3. Vũ Thành Thanh Tùng

- Viết các file `map_loader.py`, `map_info.py` và `map_check_valid.py` để đọc bản đồ, lấy vị trí các đối tượng và kiểm tra tính hợp lệ của bản đồ.
- Cài đặt năm thuật toán tìm kiếm DFS, BFS, UCS, Greedy và A* trong thư mục `src/search/`.

- Viết tài liệu hướng dẫn sử dụng `README.md` và tạo file `requirements.txt`.

Trạng thái: Hoàn thành.

2.4. Lê Anh Vũ

- Phân công công việc, theo dõi tiến độ và nhắc nhở các thành viên về thời hạn hoàn thành.
- Lập trình cơ chế di chuyển của Ghost bằng BFS trong `ghost.py`.
- Xây dựng giao diện đồ họa bằng Pygame trong thư mục `ui/`.
- Phát triển chức năng xem trước bản đồ trong `map_browser.py` và sửa lỗi giao diện bị khựng khi chạy ở 60 FPS.

Trạng thái: Hoàn thành.

3. Các công việc phối hợp nhóm

Ngoài nhiệm vụ cá nhân, các thành viên phối hợp trong quá trình tích hợp chương trình, kiểm tra kết quả và sửa lỗi.

3.1. Xử lý Ghost đi trùng vị trí và tích hợp tìm đường

Thành viên tham gia: Vũ Thành Thanh Tùng, Bùi Chí Thành.

- Cùng nghiên cứu cơ chế phân tách hướng di chuyển, hạn chế việc các Ghost đi đè lên nhau khi đuổi theo Pacman.
- Kết nối phần đọc bản đồ `map_loader.py`, trạng thái trò chơi `game_state.py` với các thuật toán tìm kiếm và chương trình chính `main.py`.

3.2. Tìm kiếm đối kháng và điều chỉnh heuristic

Thành viên tham gia: Khúc Hữu Trung, Bùi Chí Thành,Vũ Thành Thanh Tùng

- Điều chỉnh hàm đánh giá trong `adv_problem.py` và kiểm tra cách cập nhật alpha tại nút gốc của Alpha-Beta Pruning khi sử dụng điểm phạt quay lại ô đã đi qua.
- Chạy thử để xem xét hành vi của Ghost khi chặn đường hoặc bám theo Pacman.

3.3. Tính khoảng cách trong mê cung và viết cơ sở lý thuyết

Thành viên tham gia: Khúc Hữu Trung, Vũ Thành Thanh Tùng.

- Xây dựng hàm `_compute_bfs_distances` để tính khoảng cách trong mê cung, phục vụ việc đánh giá trạng thái trong `adv_problem.py`.
- Phối hợp viết phần cơ sở lý thuyết về các thuật toán trong báo cáo `report.pdf`.

3.4. Kết hợp thuật toán đối kháng với chuyển động của Ghost

Thành viên tham gia: Khúc Hữu Trung, Lê Anh Vũ.

- Kiểm tra hoạt động của Minimax và Alpha-Beta Pruning khi kết hợp với cơ chế di chuyển của Ghost trên giao diện mô phỏng.
- Đối chiếu kết quả thuật toán với chuyển động thực tế để phát hiện và sửa lỗi.

3.5. Xây dựng bản đồ, kiểm thử và sửa lỗi

Thành viên tham gia: Toàn bộ nhóm.

- Tạo và chạy thử hơn 20 bản đồ trong quá trình phát triển, sau đó chọn lọc 10 bản đồ chuẩn hóa đưa vào thư mục `maps/`.
- Cùng kiểm tra chương trình và sửa các lỗi phát sinh, chẳng hạn như Pacman đến Goal khi chưa ăn hết Food hoặc giao diện bị khựng khi chạy.

4. Tuyên bố sử dụng trí tuệ nhân tạo (AI)

Nhóm sử dụng ChatGPT, Gemini và Claude để tra cứu cú pháp Python, tham khảo cách triển khai một số thành phần và hỗ trợ tìm nguyên nhân gây lỗi trong quá trình làm đồ án.

4.1. Phạm vi sử dụng AI

Nhóm nghiên cứu và cài đặt các thuật toán tìm kiếm DFS, BFS, UCS, Greedy, A*, Multi-Goal Search, Minimax và Alpha-Beta Pruning dựa trên kiến thức môn học và tài liệu tham khảo. Các gợi ý từ AI được xem xét, trao đổi với các thành viên còn lại trong nhóm và điều chỉnh theo cấu trúc dự án.

4.2. Các nội dung có tham khảo AI

- **a. Tổ chức chương trình theo OOP trong Python (Claude):** Do các thành viên đều chuyển từ nền tảng OOP của C++ sang Python, nhóm có tham khảo cách định nghĩa các phương thức đặc thù như `__hash__`, `__eq__`, cách tổ chức lớp `GameState` và phân chia mã nguồn giữa `src/` và `ui/`.
- **b. Cải thiện heuristic cho Multi-Goal Search (Gemini):** Tham khảo hướng tiếp cận sử dụng cây khung nhỏ nhất (MST) cho bài toán nới lỏng, sau đó tự cài đặt thuật toán Prim và bộ nhớ đệm `_mst_cache`, giúp giảm khoảng 51% số node mở rộng trong thực nghiệm.
- **c. Xử lý hành vi của Ghost (Gemini và Claude):** Tham khảo các hướng xử lý Ghost di chuyển lặp hoặc trùng vị trí, sau đó kết hợp BFS với cơ chế lọc hành động an toàn `safe_actions`.
- **d. Tìm và sửa lỗi chương trình (ChatGPT và Claude):** Tham khảo cách xác định nguyên nhân giao diện bị khựng do khởi tạo font liên tục, rà soát logic cập nhật alpha trong Alpha-Beta Pruning và kiểm tra điều kiện thắng khi bản đồ có cả Food và Goal.
- **e. Công cụ chạy thử và kiểm thử (Gemini và ChatGPT):** Tham khảo cách sử dụng `argparse`, `time` và `unittest` để thiết lập tham số chạy chương trình, đo thời gian thực thi và kiểm tra kết quả thuật toán.
- **f. Thiết kế và xây dựng giao diện đồ họa (Claude và Gemini):** Nhóm tham khảo Claude về cách tổ chức các thành phần giao diện bằng Pygame, bao gồm module renderer, bố cục bảng điều khiển và phối màu. Với các phần kỹ thuật như căn chỉnh tọa độ hiển thị, tích hợp luồng xử lý và xây dựng bộ xem trước bản đồ trong map_browser.py, tham khảo hướng dẫn từ Gemini, sau đó đối chiếu với tài liệu chính thức của Pygame, kiểm thử và điều chỉnh để phù hợp với thiết kế của dự án.

5. Xác nhận đóng góp của các thành viên

Các thành viên thống nhất với nội dung phân công và phần đóng góp được trình bày trong báo cáo. Mỗi thành viên chịu trách nhiệm về phần việc được giao và phối hợp với cả nhóm trong quá trình tích hợp, kiểm thử chương trình.